Transforming Perceptions: Pedagogical Efficacy of Generative Artificial Intelligence in Art-Tech Education

Mario Caudillo(✉)

Tecnologico de Monterrey, Mexico City, Mexico
mcaudillo@tec.mx

Abstract. The integration of Generative Artificial Intelligence (AI) in education has sparked transformative changes in the artistic-technological landscape, shaping both educational practices and industry perspectives. This paper deepen into the impact of integrating Generative AI within an interactive design course, focusing on digital artists' development of technical skills alongside critical thinking essential for evaluating AI-generated outputs. The study navigates challenges surrounding the ethical use of AI in creative industries, addressing concerns about copyright and fostering responsible AI deployment.

Professional-grade tools including ChatGPT 3.5, Unity game engine version 2022.3, and Visual Studio Code with C# plugins were utilized, ensuring a robust development process. The research methodology followed action research principles, emphasizing collaboration and iterative refinement. Data collection combined Likert scale and open-ended questions, enabling a comprehensive analysis capturing both quantitative trends and qualitative insights.

Results indicate a significant shift in students' perceptions, with 60% expressing positivity towards programming post-course, and 90% viewing Generative AI as a valuable tool. These findings align with previous studies highlighting AI's potential to enhance computational thinking and problem-solving skills. However, the study acknowledges limitations due to sample size constraints, emphasizing the need for replication in larger cohorts for comprehensive validation.

The study's implications extend to AI literacy integration, metacognitive strategies, and prompt writing skills development in educational frameworks, facilitating optimal AI tool utilization. By reshaping negative perceptions and fostering critical thinking, this research contributes to bridging the perception-action gap and propels education and creative industries towards effective AI integration, innovation, and growth.

Keywords: educational innovation · higher education · professional education · AI in education

1 Introduction

As we venture into the dynamic realm of technology, Generative Artificial Intelligence (AI) is becoming a transformative force in artistic-technological fields, profoundly influencing education and industry practices [3]. This shift is particularly evident in our interactive design course, where digital artists engage with programming fundamentals to

T. Schlippe et al. (Eds.): AIET 2024, LNDECT 228, pp. 372–386, 2025.
https://doi.org/10.1007/978-981-97-9255-9_25

not only hone technical skills but also nurture critical thinking crucial for evaluating AI-generated outputs.

The course's overarching objective extends beyond technical mastery; it aims to equip students with disciplinary competencies in digital art, including strategic preproduction development, workflow optimization, efficient production pipeline execution, and innovative prototype creation. This comprehensive approach demands a holistic understanding within a condensed timeframe, emphasizing the need for adaptable educational frameworks to keep pace with rapid technological advancements.

However, the swift integration of Generative AI in the artistic realm brings challenges, notably concerning policies on copyrighted material use for AI training and ethical considerations [3]. These concerns are reflected in recent discussions among industry professionals, highlighting the negative perception and underutilization of Generative AI despite its potential benefits [3].

Our research endeavors to bridge this perception-action gap by empowering future art-technology professionals with essential AI skills, fostering critical thinking, and reshaping perceptions on responsible Generative AI deployment [1]. Through a thorough exploration of current perceptions and potential barriers, we aim to align education and industry practices, propelling students and the creative industry toward a harmonious integration of Generative AI technologies that fosters innovation and sustainable growth.

2 Materials and Methods

2.1 Description of Materials

The implementation utilized several professional-grade tools and technologies to ensure a robust and efficient development process. Specifically, we employed ChatGPT 3.5, a cutting-edge language model, for natural language processing tasks and interactions within the project. Complementing this, we utilized the Unity game engine version 2022.3, renowned for its capabilities in creating immersive and interactive experiences, providing a solid foundation for game development. For code development and editing, we relied on Visual Studio Code, a versatile and powerful code editor, enhanced with plugins tailored for C# code editing specifically designed for Unity, ensuring streamlined and effective coding practices throughout the project lifecycle. This selection of tools and technologies reflects a professional approach aimed at achieving high-quality results and optimizing development workflows.

2.2 Experimental Design

The experimental design adhered to the action research methodology, renowned for its systematic and iterative framework, fostering collaboration between researchers and participants to address real-world challenges and drive meaningful change. This approach was pivotal in understanding the complexities of integrating Generative AI within the creative industry and educational environments.

The implementation spanned five weeks and was conducted on-campus with a low student enrollment, allowing for the inclusion of only one group this spring semester,

comprising 24 students. This study represents the first cycle of an ongoing series, with subsequent cycles planned for future periods to ensure continuous refinement and improvement.

Each cycle of the action research methodology was divided into four distinct phases:

Planning: This initial phase involved designing the curriculum and selecting appropriate Generative AI tools for integration into the course. Objectives were set to enhance students' technical skills and critical thinking abilities. Detailed lesson plans were developed, incorporating both theoretical and practical components to provide a comprehensive learning experience.

Action: In this phase, the planned curriculum was implemented. Students engaged with Generative AI tools through hands-on activities and collaborative projects. Instructors provided continuous support and guidance, ensuring that students could effectively use the tools and apply their knowledge in creative contexts.

Observation: Throughout the course, data was collected to monitor students' progress and perceptions. This included surveys, interviews, and classroom observations. The small group involved in this initial cycle was informed about the importance of sincerity in their responses to minimize biases. Observations focused on students' engagement, their ability to integrate AI tools into their projects, and changes in their attitudes toward AI.

Reflection: After the course, the collected data was analyzed to evaluate the effectiveness of the intervention. Quality control measures were implemented during data analysis to ensure accuracy and reliability. These measures included cross-checking data entries, verifying survey responses, and triangulating data from multiple sources to draw comprehensive insights. Reflections from both students and instructors were gathered to identify areas for improvement and to inform the planning of future cycles.

This detailed and iterative approach ensures that the integration of Generative AI into the course is continually refined, based on feedback and outcomes from each cycle. The ongoing cycles will allow for the adaptation of strategies and methodologies, promoting the sustainable and effective incorporation of AI technologies in art-tech education. Through this iterative process, we aim to develop a robust educational framework that prepares students to effectively leverage Generative AI in their creative and professional endeavors.

2.3 Data Collection Procedures

To gather comprehensive data, we employed a mixed-methods approach combining quantitative and qualitative research instruments. This included:

Surveys: Pre-course and post-course surveys were administered to assess changes in students' perceptions and attitudes toward programming and AI. The surveys included Likert-scale questions, multiple-choice questions, and open-ended questions to capture nuanced responses.

Interviews: Semi-structured interviews with selected students provided deeper insights into their experiences and perceptions. This method allowed for follow-up questions and clarification, facilitating a richer understanding of the participants' views.

Observations: Classroom observations were conducted to monitor student engagement and interaction with AI tools. Observers used a standardized checklist to ensure consistency in data collection.

2.4 Data Analysis Method

The data analysis methodology employed a comprehensive approach, combining quantitative analysis based on Likert scale questions with qualitative analysis of open-ended questions. This multifaceted approach allowed for a thorough examination of the data, capturing both numerical trends and nuanced insights from participants' detailed responses. The Likert scale provided a structured framework for quantifying participants' attitudes and opinions, facilitating the identification of patterns and trends across the data set. Simultaneously, the qualitative analysis of open-ended questions conducted investigations into deeper into participants' perspectives, uncovering rich qualitative data that added depth and context to the quantitative findings. By integrating quantitative and qualitative methods, the data analysis process ensured a holistic understanding of the research subject, enabling nuanced interpretations and robust conclusions.

2.5 Quality Control and Validation

The validation process for data collection instruments was rigorously carried out to safeguard against potential biases that could undermine the integrity of the study's findings. This critical step involved meticulously assessing the reliability and validity of the data collection tools to ensure their effectiveness in capturing accurate and unbiased data. By meticulously validating the instruments, researchers could confidently rely on the collected data as a robust foundation for analysis and interpretation, enhancing the credibility and trustworthiness of the study's outcomes.

Quality control during data collection and analysis was a critical aspect of this study. To maintain high standards, several steps were taken:

Data Collection: Students were instructed on the importance of providing honest feedback. Surveys and interviews were designed to be straightforward and unambiguous to reduce the risk of misinterpretation.

Data Analysis: Rigorous procedures were followed to ensure data integrity. Responses were anonymized to encourage openness, and multiple researchers independently reviewed the data to mitigate bias. Statistical analysis was performed to identify significant trends and correlations.

2.6 Limitations

The primary limitation of the study stems from the relatively modest sample size of 24 students, which was constrained by logistical considerations. This constraint underscores the need for future replication studies involving larger cohorts to enhance the generalizability and statistical power of the findings. Expanding the participant pool would enable researchers to capture a more comprehensive range of perspectives and experiences, thereby strengthening the study's validity and reliability. Additionally, a

larger sample size would facilitate more robust statistical analyses and contribute to a deeper understanding of the phenomena under investigation, offering valuable insights for future research endeavors.

This course each spring semester open a total of 14 groups nationwide, with 15–30 students, which gave the opportunity to expand the participation pool in the next years if the instructors are interested in collaborate with this research.

2.7 Reproducibility

During the first week of classes, students were introduced to foundational algorithm concepts, including constants, variables, and data types. They engaged in decision-making processes and learned about classes, objects, attributes, and methods essential for programming. Additionally, they received an overview of human-computer interaction principles and user interface design fundamentals. The mastery activity for this week consisted in creating an specific code based on the algorithm stated in Code 1.

```
variables: point1X, point1Y, point2X, point2Y
In the first frame: Initialize point1X and point1Y to 5,
point2X and point2Y to 4. Print the values of the varia-
bles separated by a space and create a variable that
stores the distance between the two points. Create a var-
iable that stores the angle between these two points.
Print the values of the variables on one line, on the
next line, print the results obtained separated by a
space.
Example: (point1X) (point1Y) (point2X) (point2Y) (Dis-
tance) (Angle)
Example with values:
5 5 4 4
1.414214 -135
Test it with values: 2 2 4 4 should result in 2.828427 45
Make sure your formula works by manually verifying the
written code.
Test case input: 5 5 4 4 with expected result: 1.414214 -
135 Test case input: 2 2 4 4 with expected result:
2.828427 45 Test case input: 0 0 1 0 with expected re-
sult: 1 0 Test case input: 0 0 0 1 with expected result:
1 90 Test case input: 0 0 -1 0 with expected result: 1
180 Test case input: 0 0 0 -1 with expected result: 1 -90
Test case input: 2 2 -4 4 with expected result: 6.324555
161.565 Test case input: 2 2 4 -4 with expected result:
6.324555 -71.56505 Test case input: 6 5 4 3 with expected
result: 2.828427 -135 Test case input: 3 4 5 6 with ex-
pected result: 2.828427 45
```

Code 1. Algorithm for the Mastery activity in week 1.

As illustrated in the Code 2 is the code generated by the students with professor's guidance.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class PM1 : MonoBehaviour
{
public int point1X;
public int pointlY;
public int point2X;
public int point2Y;
void Start()
{
float distance;
float angle;
distance=Mathf.Sqrt (Mathf.Pow(point2X-point1x,
2)+Mathf.Pow(point2Y-point1Y, 2));
angle=Mathf.Atan2(point2Y-point1Y,point2X-point1X)*
Mathf.Rad2Deg;
Debug.Log(point1X+" "+pointlY+" "+point2X+"
"+point2Y+"\n" + distance + " " + angle);}}
```

Code 2. Final code developed by students with professor's guidance.

In the second week, students deepen into programming concepts, focusing on functions, String, and loops to enhance their coding skills. They explored Object-Oriented Design principles, mastering class definition and instantiation techniques; they developed from idea UML diagrams as use cases, class diagram and state machine tables. Moreover, students learned about GUI Layouts, setting the stage for the creation of interactive user interfaces. The culmination of this week's activities was the development of a clicker game, where students could apply their newfound skills and creativity.

As an introductory phase to the utilization of generative artificial intelligence, supplementary activities were incorporated during the second week of the clicker game project. These activities were not part of the regular class curriculum but were provided as instructions to students on how they could utilize tutorials or ChatGPT to gain insights into solving the requested problems. Additional game mechanics were introduced in the clicker game, including the implementation of a leaderboard displaying the top 5 scores and the integration of an audio source that persists across scenes. These supplementary activities served as examples of leveraging external resources and expanding the functionality of the game project.

Transitioning into the third week, students were introduced to Generative AI, marking a significant milestone in their educational journey. Code 3–5 illustrates the multiple prompt attempts of students to get what was needed in week 1 assignment, this showed the integration of Generative AI prompts into the learning environment, emphasizing the importance of specificity for optimal results.

```
Student generated prompt: Which operation using the Mathf
library in Unity could I use to get the angle in degrees
between two points stored as coordinates (point1X,
point1Y) and (point2X, point2Y)?
ChatGPT response: You can use the Atan2 function from the
Mathf class in Unity to calculate the angle between two
points given their x and y components. Here's an example
of how you could do it:
using UnityEngine;

public class AngleCalculator : MonoBehaviour
{
    public Vector2 point1;
    public Vector2 point2;

    void Start()
    {
        // Calculate the difference in coordinates be-
tween the two points
        Vector2 direction = point2 - point1;
        // Calculate the angle in radians using Atan2
        float angleRadians = Mathf.Atan2(direction.y, di-
rection.x);
        // Convert the angle from radians to degrees
        float angleDegrees = angleRadians *
Mathf.Rad2Deg;
        // Print the resulting angle
        Debug.Log("Angle between the two points: " + an-
gleDegrees + " degrees.");
    }
}
In this script, 'point1' and 'point2' are the two points
whose angle you want to calculate. The difference between
'point2' and 'point1' gives the direction from 'point1'
to 'point2'. Then, Mathf.Atan2 calculates the angle in
radians between this direction and the x-axis. Finally,
the angle is converted to degrees using Mathf.Rad2Deg.
```

Code 3. Student generated prompt: Which operation using Mathf library in unity helps us get the angle between two points saved as coordinates (point1X,point1Y) and (point2X,point2Y). Resulted on the creation of Vector2 variables which have the coordinates.

```
ChatGPT response: Here is a Python implementation of the
algorithm you described:

import math

# Initialize variables
point1X = 5
point1Y = 5
point2X = 4
point2Y = 4

# Print the initial values of the variables
print(point1X, point1Y, point2X, point2Y)

# Calculate the distance between the two points
distance = math.sqrt((point2X - point1X)**2 + (point2Y -
point1Y)**2)

# Calculate the angle between the two points
angle = math.atan2(point2Y - point1Y, point2X - point1X)
* 180 / math.pi

# Print the results
print(distance, angle)
This code initializes the variables point1X, point1Y,
point2X, and point2Y with the given values. It then cal-
culates the distance between the two points using the Eu-
clidean distance formula and the angle between the two
points using the atan2 function and converting it from
radians to degrees. Finally, it prints the values of the
variables followed by the distance and the angle.
```

Code 4. Response for second student generated prompt: Develop the following algorithm, copied from Code 1. Result a code developed in python because of the lack of specificity in prompt creation.

```
ChatGPT response: Here is an implementation in C# using
the Mathf library in Unity:
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class PointDistanceAngle : MonoBehaviour
{
  public int point1X;
  public int pointlY;
  public int point2X;
  public int point2Y;
  void Start()
  {
    float distance;
    float angle;
    distance=Mathf.Sqrt (Mathf.Pow(point2X-point1x,
2)+Mathf.Pow(point2Y-point1Y, 2));
    angle=Mathf.Atan2(point2Y-point1Y,point2X-point1X)*
Mathf.Rad2Deg;
    Debug.Log(point1X+" "+pointlY+" "+point2X+"
"+point2Y+"\n" + distance + " " + angle);
  }
}
```

Code 5. Response for the third student generated prompt: Develop the following algorithm, copied from Code 1, using C# and library Mathf for Unity. Resulted in the activity outcome for the first week activity.

Under the guidance of the professor, students explored the potential of Generative AI as a tool for optimization in game development. The creation of a catcher game served as a tangible demonstration of the AI's capabilities, showcasing the iterative process of prompt refinement and game enhancement. Developing this game started with an explanation of what does the catcher game needs to function, which help students specify what is needed in the game scene, after working in the concept of the game and how it needs to be played, students created the next prompt for chatGPT: "Create the catcher class in C# for Unity, which moves a Unity Sprite using the right and left arrow keys, limited to staying within the screen boundaries. Create the GameManager class that displays the number of objects caught in a TMPText and holds an array of Sprites for the 3 possible failures. Create the class for the falling object that is generated with a Rigidbody and falls every 2 s to a random coordinate above the viewport." The result of this prompt is illustrated in Code 6.

```
ChatGPT response: Here is a basic implementation of the
classes you mentioned:
using UnityEngine;
using TMPro;

public class Catcher : MonoBehaviour
{
    public float speed = 5f; // Movement speed of the
catcher
    public float maxX = 8f; // Maximum limit on the X
axis
    public float minX = -8f; // Minimum limit on the X
axis

    // Update is called once per frame
    void Update()
    {
        // Move the catcher with the right and left arrow
keys
        float moveInput = Input.GetAxis("Horizontal");
        transform.Translate(Vector2.right * moveInput *
speed * Time.deltaTime);

        // Limit the movement within the screen
        Vector3 clampedPosition = transform.position;
        clampedPosition.x =
Mathf.Clamp(clampedPosition.x, minX, maxX);
        transform.position = clampedPosition;
    }
}

public class GameManager : MonoBehaviour
{
    public TMP_Text caughtText; // Reference to the
TMP_Text object to display the number of objects caught
    public Sprite[] failureSprites; // Array of Sprites
for the failures
    private int caughtCount = 0; // Counter for objects
caught

    // Method to update the counter of objects caught
    public void UpdateCaughtCount()
```

```
    {
        caughtCount++;
        caughtText.text = "Objects caught: " +
caughtCount;
    }
    // Method to display the failure sprite at the given
index
    public void ShowFailure(int index)
    {
        // Implement the logic to display the failure
sprite in the UI
        // You can use the index variable to get the
correct sprite from the failureSprites array
    }
}

public class FallingObject : MonoBehaviour
{
    public float fallSpeed = 2f; // Falling speed of the
object

    // Start is called before the first frame update
    void Start()
    {
        // Invoke the "Fall" method every 2 seconds
        InvokeRepeating("Fall", 2f, 2f);
    }

    // Method to make the object fall
    void Fall()
    {
        // Generate random coordinates above the viewport
        float randomX = Random.Range(-8f, 8f); //
Assuming the screen width is 16 units
        float randomY = 10f; // The height from where the
object will fall

        // Set the position of the object
        transform.position = new Vector2(randomX,
randomY);

        // Apply falling speed
```

```
        GetComponent<Rigidbody2D>().velocity =
Vector2.down * fallSpeed;
    }
}
Remember to attach this script to the corresponding ob-
jects in Unity for it to work properly. Make sure to con-
figure the input keys in the Unity project so that the
catcher movement works as expected.
```

Code 6. Result for prompt created by students – catcher class.

After the creation of the prompt, students started to create the unity project with the different classess, and had a sort of template in which they could continue developing the game to finish with the objective of third week mastery activity.

The fourth and fifth weeks marked the culmination of the course, where students applied their acquired knowledge and skills to develop their final 2D game projects. Throughout these weeks, students faced challenges, made breakthroughs, and honed their problem-solving abilities, resulting in the creation of innovative and engaging game prototypes. The final projects serve as a testament to the transformative impact of the course, empowering students to harness the power of technology and creativity to bring their ideas to life.

3 Results and Discussion

The implementation of Generative Artificial Intelligence (AI) in the interactive design course resulted in significant shifts in students' perceptions towards programming and AI education. Initially, 50% of students held negative opinions about programming, while the other 50% remained neutral. However, by the end of the course, 60% of students expressed positive views towards programming, 10% remained neutral, and 30% retained negative sentiments. This shift indicates a notable transformation in the way students perceive the use of generative artificial intelligence in creative industries as they gained more exposure and hands-on experience with AI tools [3].

Similarly, attitudes towards the use of AI in the creation of digital art and videogames underwent a substantial change throughout the course. At the outset, 70% of students had negative perceptions, viewing AI as a form of cheating, plagiarism, or harboring fears about its usage. In contrast, 30% were neutral, either due to a lack of familiarity or previous exposure to Generative AI. However, as the course progressed and students engaged with Generative AI in various activities, their perceptions evolved. By the course's conclusion, only 20% of students opposed Generative AI's use, while a significant majority of 80% embraced positive perceptions, appreciating how Generative AI helped them explore new topics, optimize workflows, and adapt their learning experiences to be more dynamic and engaging [3].

These findings are consistent with previous studies demonstrating the positive impact of AI tools on students' computational thinking skills [6]. Lin and Chen observed significantly higher computational thinking skills in students using deep learning recommendation-based systems, highlighting the potential of Generative AI to enhance problem-solving abilities and critical thinking skills among learners. Similarly, García

et al. [5] noted improvements in students' computational thinking skills with AI training, further emphasizing the pedagogical value of integrating Generative AI technologies into educational contexts [4].

Moreover, when asked about Generative AI's influence on their technology interaction, a remarkable 90% of students viewed it as a useful tool, shifting from seeing AI as an adversary or a cheating method to recognizing it as a valuable resource for creativity and innovation [3]. Additionally, 70% reported that Generative AI significantly enhanced their learning experiences, providing them with new insights, perspectives, and opportunities to experiment and iterate in their projects. Although 20% remained neutral, indicating varying levels of comfort or familiarity with Generative AI, only a minimal 10% abstained from its use altogether, highlighting the widespread acceptance and adoption of Generative AI technologies among students in the course [3].

As highlighted in Yilmaz and Karaoglan's research [7], effective utilization of AI tools like Generative AI requires students to develop prompt writing skills and a deep understanding of how to leverage AI effectively in their projects [2]. Teachers play a crucial role in fostering AI literacy and metacognitive strategies, empowering students to think critically, analyze AI-generated outputs, and evaluate their learning processes when using AI tools. Metacognitive prompts serve as guides for students, enabling them to direct their learning effectively, leading to improved understanding, control, and regulation of their educational experiences. Overall, the integration of Generative AI in the interactive design course had a profound impact on students' perceptions, learning outcomes, and readiness to embrace Generative AI technologies in their future endeavors.

4 Conclusion

The study's exploration into the integration of Generative Artificial Intelligence (AI) in educational settings, specifically in an interactive design course for digital artists, has yielded significant insights and outcomes. One of the major outcomes of this work is the noticeable shift in students' perceptions towards programming and the use of AI in creative industries as a tool to an effective pipeline. The initial distribution of opinions, with 100% expressing not possitivity towards programming and AI, underwent a transformative change, with 60% now holding positive views towards programming and a remarkable 90% considering Generative AI a valuable tool for technology interaction. This shift is crucial as it signifies a move from apprehension and skepticism to acceptance and appreciation of Generative AI technologies, highlighting the potential for Generative AI to enhance learning experiences and foster exploration in creative domains.

Furthermore, the study emphasizes the importance of AI literacy and metacognitive strategies in effectively utilizing AI tools like Generative AI. Teachers play a pivotal role in guiding students to develop prompt writing skills and critical thinking abilities necessary for leveraging AI tools optimally. The findings align with previous research indicating the positive impact of AI training on students' computational thinking skills, underscoring the relevance of integrating the correct and ethical usage of AI in curricula.

Despite these significant outcomes, the study acknowledges certain limitations. The relatively small sample size of 24 students necessitates replication in larger cohorts to

validate the findings comprehensively. Moreover, the study's focus on a specific course and educational context may limit the generalizability of results to broader educational settings.

The relevance of this work lies in its contribution to bridging the perception-action gap regarding Generative AI's responsible use and its pedagogical utility in programming education. By reshaping negative perceptions, enhancing learning outcomes, and fostering critical thinking, this study paves the way for future educational practices that embrace AI technologies effectively.

In practical terms, the study's recommendations include the integration of AI literacy skills and metacognitive strategies in educational frameworks, along with the development of prompt writing skills among students. This holistic approach can maximize the benefits of AI tools like Generative AI, preparing students for AI-enriched environments and facilitating innovation in creative industries.

This study underscores the transformative potential of Generative AI in arts and technology education and its pivotal role in shaping future professionals' skills and perceptions. Through continued research, collaboration, and adaptation of educational strategies, the seamless integration of AI technologies can be achieved, heralding a new era of innovation and growth in educational practices and creative industries alike.

Acknowledgements. I want to acknowledge the technical and financial support of the Writing Lab, Institute for the Future of Education, Tecnológico de Monterrey, Mexico, in the production of this work.

Declaration of Interest Statement. The author declare that they have no conflict of interests.

References

1. Chiu, T.K.F.: Future research recommendations for transforming higher education with generative AI. Comput. Educ. Artif. Intell. **6**, 100197 (2024). ISSN 2666–920X, https://doi.org/10.1016/j.caeai.2023.100197
2. Dunder, N., Lundborg, S., Wong, J., Viberg, O.: Kattis vs ChatGPT: Assessment and evaluation of programming tasks in the age of artificial intelligence. In: LAK '24: Proceedings of the 14th Learning Analytics and Knowledge Conference, pp. 821–827 (2024). https://doi.org/10.1145/3636555.3636882
3. Johnston, H., Wells, R.F., Shanks, E.M., et al.: Student perspectives on the use of generative artificial intelligence technologies in higher education. Int. J. Educ. Integr. **20**, 2 (2024). https://doi.org/10.1007/s40979-024-00149-4
4. Knoth, N., Tolzin, A. , Janson, A., Leimeister, J.M.: AI literacy and its implications for prompt engineering strategies. Comput. Educ. Artif. Intell. **6**, 100225 (2024). ISSN 2666–920X, https://doi.org/10.1016/j.caeai.2024.100225
5. García, J.D.R., et al.: Developing computational thinking at school with machine learning: an exploration. In: 2019 International Symposium on Computers in Education (SIIE), pp. 1–6. IEEE (2019)

6. Lin, P.H., Chen, S.Y.: Design and evaluation of a deep learning recommendation based augmented reality system for teaching programming and computational thinking IEEE. Access **8**, 45689–45699 (2020)
7. Yilmaz, R., Karaoglan, F.: The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. Comput. Educ. Artif. Intell. **4**, 100147 (2023). ISSN 2666–920X, https://doi.org/10.1016/j.caeai.2023.100147